SIMATS ENGINEERING

JAVA PROGRAMMING

ASSIGNMENT – 2

Java Strings and Collection Framework

Student Details	Information
Student Name	Dhanuja D
Register Number	192311427
Department	Computer Science and Engineering
Institution	SIMATS Engineering
Course	Java Programming
Assignment	Assignment 2 – CO2
Total Marks	50
Submission Date	02/11/26

DECLARATION

I hereby declare that this assignment report titled “Java Strings and Collection Framework” is prepared by me as part of the Java Programming course. The programs, observations and explanations are intended for learning and academic evaluation. I will execute the programs, verify the outputs and enter my own measured experimental results before final submission.

Student Signature: DHANUJA

Date: 2/11/26

ACKNOWLEDGEMENT

I express my sincere thanks to the faculty members of SIMATS Engineering for providing the assignment and guidance to understand Java Strings, collections, generics, sorting and queue-based data structures. I also thank the institution for providing the programming laboratory facilities required to implement and test the programs.

TABLE OF CONTENTS

Section	Title
1	Introduction and objectives
2	Q1 – Building Text Under Load: String, StringBuilder and StringBuffer
3	Q2 – ArrayList, LinkedList and ArrayDeque
4	Q3 – HashSet and the hashCode–equals contract
5	Q4 – HashMap, TreeMap and LinkedHashMap
6	Q5 – Generics, Comparable and Comparator
7	Q6 – Queue, PriorityQueue and fairness
8	Q7 – Diagnose, redesign and validate
9	Overall conclusion
10	Execution and GitHub submission checklist

1. INTRODUCTION AND OBJECTIVES

The Java Collections Framework provides reusable data structures for storing, accessing and processing groups of objects. Different structures have different strengths. A correct engineering choice depends on the operations performed by an application, the amount of data and the required ordering.

This assignment studies Java String handling and the Collection Framework through practical application examples: railway e-ticket generation, a music playlist, event attendance, pharmacy inventory, a sports leaderboard and hospital emergency triage.

1.1 Objectives

- Understand the difference between immutable String and mutable StringBuilder/StringBuffer.
- Compare ArrayList, LinkedList and ArrayDeque for common list and deque operations.
- Understand why equals() and hashCode() must be consistent when objects are stored in hash-based collections.
- Compare HashMap, TreeMap and LinkedHashMap for lookup and iteration order.
- Apply generics and Comparator chains to create reusable ranking logic.
- Use PriorityQueue to process patients by severity while preserving arrival order for ties.
- Diagnose a performance bottleneck, redesign the implementation and validate the change with evidence.

1.2 Development environment

Item	Details to record
Operating system	Windows / Linux / macOS: _____
Java version	Run: java -version; record result: _____
IDE	Eclipse / IntelliJ IDEA / VS Code / command line
Processor and RAM	Record your computer specifications
Measurement method	System.nanoTime(), repeated runs and average time

2. Q1 – BUILDING TEXT UNDER LOAD: STRING, STRINGBUILDER OR STRINGBUFFER

2.1 Problem description

A railway reservation system creates a printable e-ticket by joining PNR, passenger name, coach, berth, fare and journey details. The current approach uses the + operator repeatedly inside a loop. Since String objects are immutable, repeated concatenation can create many temporary objects.

2.2 Concepts

Class	Meaning	Suitable use
String	Immutable character sequence. A change creates a new String.	Fixed text and values that should not change.
StringBuilder	Mutable character sequence; not synchronized.	Fast text construction in a single thread.
StringBuffer	Mutable and synchronized.	Shared mutable text where thread safety is required.

2.3 Algorithm

1. Read the number of ticket fields and the number of tickets.
2. Build ticket text using String concatenation, StringBuilder and StringBuffer.
3. Repeat each workload several times to reduce timing noise.
4. Measure elapsed time with System.nanoTime().
5. Compare the average times and explain object creation and thread-safety differences.

2.4 Java program

```
public class TicketTextBenchmark {
    static String buildWithString(int fields) {
        String ticket = "";
        for (int i = 1; i <= fields; i++) {
            ticket = ticket + "Field" + i + ":Value" + i + "\n";
        }
        return ticket;
    }

    static String buildWithBuilder(int fields) {
        StringBuilder ticket = new StringBuilder();
```

```
        for (int i = 1; i <= fields; i++) {
            ticket.append("Field").append(i)
                  .append(":Value").append(i).append('\n');
        }
        return ticket.toString();
    }

    static String buildWithBuffer(int fields) {
        StringBuffer ticket = new StringBuffer();
        for (int i = 1; i <= fields; i++) {
            ticket.append("Field").append(i)
                  .append(":Value").append(i).append('\n');
        }
        return ticket.toString();
    }

    static long measure(Runnable task, int repetitions) {
        long start = System.nanoTime();
        for (int i = 0; i < repetitions; i++) task.run();
        return (System.nanoTime() - start) / repetitions;
    }

    public static void main(String[] args) {
        int fields = 50;
        int repetitions = 2000;
        System.out.println("String average ns: " +
            measure(() -> buildWithString(fields), repetitions));
        System.out.println("StringBuilder average ns: " +
            measure(() -> buildWithBuilder(fields), repetitions));
        System.out.println("StringBuffer average ns: " +
            measure(() -> buildWithBuffer(fields), repetitions));
        System.out.println("Ticket preview:\n" + buildWithBuilder(5));
    }
}
```

2.5 Expected behaviour and analysis

All three methods should produce equivalent ticket text. StringBuilder is generally expected to perform well for repeated concatenation in a single thread because it modifies a resizable character buffer. StringBuffer provides synchronized methods and may have extra synchronization overhead. Repeated String concatenation may be slower because intermediate immutable objects can be created.

Workload	String average (ns)	StringBuilder average (ns)	StringBuffer average (ns)
50 fields × repeated runs	Fill after execution	Fill after execution	Fill after execution
Additional workload of your choice	Fill after execution	Fill after execution	Fill after execution

Conclusion for Q1: For local, single-threaded ticket construction, StringBuilder is normally the preferred choice. StringBuffer is considered when the same mutable buffer must be accessed by multiple threads with synchronization. String remains suitable for fixed or small text.

2.6 Additional question: shared booking log

For each individual ticket, StringBuilder is suitable because each thread can build its own ticket locally. For a shared log accessed by multiple threads, use a thread-safe design such as synchronized access to a shared StringBuffer or, preferably, a logging framework / synchronized queue. StringBuffer alone does not make a multi-step business operation atomic.

2.7 Complexity

For n appended characters, StringBuilder and StringBuffer provide approximately O(n) total append work (amortized). Repeated immutable concatenation in a loop can copy an increasing amount of previous text and may approach $O(n^2)$ total copying for similarly sized appended fragments.

3. Q2 – WHEN IS ARRAYLIST AN APPROPRIATE ENGINEERING CHOICE?

3.1 Problem description

A music playlist allows users to add songs at the end, insert or remove songs at selected positions, access the k-th song and iterate through all songs. The assignment asks for a comparison of ArrayList and LinkedList, and a changed condition in which ArrayDeque is used for an Up Next queue.

3.2 Comparison

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	Amortized O(1)	O(1) with tail link	Amortized O(1)
Insert/remove at front	O(n) due to shifting	O(1) at ends	O(1) at ends
Insert/remove in middle	O(n) search and shifting	O(n) to reach position; O(1) link change	Not designed for indexed middle edits
Get k-th element	O(1)	O(n)	No indexed access
Iterate	O(n), cache-friendly	O(n)	O(n)

3.3 Java program

```
import java.util.*;

public class PlaylistComparison {
    static long timeArrayList(int n) {
        ArrayList<Integer> list = new ArrayList<>();
        long start = System.nanoTime();
        for (int i = 0; i < n; i++) list.add(i);
        for (int i = 0; i < n; i++) list.get(i);
        return System.nanoTime() - start;
    }

    static long timeLinkedList(int n) {
        LinkedList<Integer> list = new LinkedList<>();
        long start = System.nanoTime();
        for (int i = 0; i < n; i++) list.add(i);
        for (int i = 0; i < n; i++) list.get(i);
        return System.nanoTime() - start;
    }
```

```
    public static void main(String[] args) {
        int[] sizes = {1000, 5000, 10000};
        for (int n : sizes) {
            System.out.println("n=" + n);
            System.out.println("ArrayList ns=" + timeArrayList(n));
            System.out.println("LinkedList ns=" + timeLinkedList(n));
        }

        ArrayDeque<String> upNext = new ArrayDeque<>();
        upNext.addLast("Song A");
        upNext.addLast("Song B");
        upNext.addFirst("Priority Song");
        System.out.println("Next song: " + upNext.removeFirst());
    }
}
```

3.4 Test cases and observations

Test	Input size / operation	Expected result
1	Add 1,000 songs and access each index	All songs are accessible in insertion order.
2	Add 5,000 songs and access each index	Record elapsed time for both lists.
3	Add 10,000 songs and access each index	Record elapsed time for both lists.
4	Insert at both ends of ArrayDeque	First item removed should be Priority Song.

n	ArrayList time (ns)	LinkedList time (ns)	Observed faster structure
1,000	Fill after execution	Fill after execution	Fill after execution
5,000	Fill after execution	Fill after execution	Fill after execution
10,000	Fill after execution	Fill after execution	Fill after execution

3.5 Analysis and conclusion

ArrayList is a reasonable choice when the application frequently reads items by index, appends at the end and iterates over the playlist. It is less suitable when frequent insertions and removals occur near the front. LinkedList supports efficient changes at its ends, but indexed access is slow because it must traverse nodes. ArrayDeque is

appropriate for an Up Next queue when songs are added at both ends and removed from the front.

The benchmark above measures append plus indexed access. It does not prove that one structure is fastest for every operation. Separate tests should be performed for front insertion, middle insertion, removal and iteration.

4. Q3 – ENGINEER THE hashCode–equals CONTRACT

4.1 Problem description

A college festival scans student ID cards at several gates. The same student may be scanned more than once. A HashSet<Student> should store only one entry for each student registration number.

4.2 Important rule

If two objects are equal according to equals(), they must return the same hashCode(). Different objects may have the same hash code, but equal objects must never have different hash codes. HashSet uses hashCode() to find a bucket and equals() to check equality within that bucket.

4.3 Java program

```
import java.util.*;

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Student)) return false;
        Student other = (Student) obj;
        return Objects.equals(regNo, other.regNo);
    }

    @Override
```

```
    public int hashCode() {
        return Objects.hash(regNo);
    }

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}

public class AttendanceDemo {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));
        System.out.println("Unique students: " + present.size());
        for (Student s : present) System.out.println(s);
    }
}
```

4.4 Test design

Test	Data	Expected set size
1 – Duplicate scan	Arun / 192311001 added twice	1 for Arun
2 – Different student	Divya / 192311002 added	2 total
3 – Same ID, different name	Same registration number with changed display name	Still one entry for that ID

4.5 Before and after comparison

Before implementing hashCode(), two Student objects with the same registration number may be placed in different hash buckets because the default Object.hashCode() usually differs by object identity. The set may then contain duplicates even though equals() says the objects are equal. After overriding hashCode() using the same equality field (regNo), duplicate scans are recognised.

Version	Observed unique count (record)	Explanation

Without hashCode()	Record actual result	Possible duplicate because hash buckets differ.
With matching hashCode()	Expected: 2 for the supplied program	Same ID objects have same hash and compare equal.

4.6 Alternative designs

A TreeSet with a Comparator based on registration number can also prevent duplicates, generally with O(log n) insertion and lookup. A HashMap<String, Student> keyed by registration number is another option. HashSet commonly provides expected O(1) add and contains operations, while TreeSet provides sorted order.

Conclusion for Q3: equals() and hashCode() must use the same identity fields. The registration number is selected as the unique identity of a student. Fields used for hashing should not be changed while the object is stored in a HashSet.

5. Q4 – CHOOSING A MAP FOR A PHARMACY INVENTORY

5.1 Problem description

A hospital pharmacy stores medicine stock using a unique medicine code. Pharmacists perform frequent lookups. At the end of the day, the system prints one report in medicine-code order and another report in the order medicines were added.

5.2 Map comparison

Map	Typical get/put	Iteration order	Best use
HashMap	Expected O(1)	No guaranteed order	Fast key lookup when order is not important.
TreeMap	O(log n)	Sorted by key	Naturally produces medicine-code order.
LinkedHashMap	Expected O(1)	Insertion order by	Naturally produces

		default	order-added report.

5.3 Java program

```
import java.util.*;

public class PharmacyInventory {
    public static void main(String[] args) {
        Map<String, Integer> hash = new HashMap<>();
        Map<String, Integer> tree = new TreeMap<>();
        Map<String, Integer> linked = new LinkedHashMap<>();

        String[] codes = {"M103", "M101", "M105", "M102", "M104"};
        int[] stocks = {40, 25, 60, 30, 15};

        for (int i = 0; i < codes.length; i++) {
            hash.put(codes[i], stocks[i]);
            tree.put(codes[i], stocks[i]);
            linked.put(codes[i], stocks[i]);
        }

        System.out.println("Lookup M103: " + hash.get("M103"));
        System.out.println("Sorted code report: " + tree);
        System.out.println("Order-added report: " + linked);
        System.out.println("HashMap report (order not guaranteed): " + hash);
    }
}
```

5.4 Test cases

Test	Operation	Expected result
1	Lookup M103	Stock value 40
2	Print TreeMap	M101, M102, M103, M104, M105
3	Print LinkedHashMap	M103, M101, M105, M102, M104
4	Print HashMap	All entries present; order is not guaranteed

5.5 Experimental table

For performance testing, repeat put/get operations with at least three inventory sizes such as 1,000, 10,000 and 50,000 medicines. Use the same generated keys and repeat each measurement. The following table must be completed from your own execution.

Number of medicines	HashMap lookup time	TreeMap lookup time	LinkedHashMap lookup time
1,000	Fill after execution	Fill after execution	Fill after execution
10,000	Fill after execution	Fill after execution	Fill after execution
50,000	Fill after execution	Fill after execution	Fill after execution

5.6 Alternative sorted-array design

If the inventory is stored in an array sorted by medicine code, binary search provides O(log n) lookup. Insertion may require shifting O(n) elements. Producing a sorted report is O(n) because the array is already sorted. A TreeMap offers O(log n) lookup and insertion and keeps keys sorted automatically. HashMap is usually preferred when lookup speed is the main priority and report order can be handled separately.

Conclusion for Q4: Use HashMap for fast unordered lookup, TreeMap for a naturally sorted code report, and LinkedHashMap for preserving insertion order.

6. Q5 – DERIVE A GENERIC RANKING COMPONENT FROM A PRINCIPLE

6.1 Problem description

An inter-college sports meet ranks teams using three rules: higher points first; if points are equal, higher goal difference first; if both are equal, team name in alphabetical order. The ranking logic should be reusable for teams, players, colleges and quiz teams.

6.2 Comparable and Comparator

Comparable defines a natural ordering inside the class through compareTo(). Comparator defines an external ordering and is useful when the same class needs multiple sorting rules. This solution uses Comparator because different leaderboards may require different ranking policies.

6.3 Java program

```
import java.util.*;

class Team {
    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    void add(T item) {
        items.add(item);
```

```
    }

    List<T> ranking() {
        List<T> result = new ArrayList<>(items);
        result.sort(order);
        return result;
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank =
            Comparator.comparingInt((Team t) -> t.points).reversed()
            .thenComparing(
                Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
            .thenComparing(t -> t.name);

        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));

        System.out.println(board.ranking());
    }
}
```

6.4 Expected output

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

6.5 Explanation

Eagles ranks first because it has 9 points and the highest goal difference of 6. Bisons and Panthers both have 9 points and a goal difference of 4, so alphabetical order places Bisons before Panthers. Cobras has only 6 points and therefore ranks last.

6.6 Correctness and complexity

The comparator checks fields in the required priority order. It compares points first, then goal difference, then name. This gives a consistent lexicographic ordering. Sorting n items takes O(n log n) time using the standard object sorting implementation. The ranking() method creates a copy, so the original insertion order in items is not changed. The copied list requires O(n) extra space.

A generic Leaderboard<T> with Comparator<? super T> can accept a comparator for T or one of its supertypes. This makes the component reusable without forcing every domain class to implement Comparable.

7. Q6 – PRIORITY, ORDER AND FAIRNESS IN A QUEUE

7.1 Problem description

A hospital emergency department assigns each patient a severity number from 1 (critical) to 5 (minor). The patient with the lowest severity number must be treated first. If two patients have the same severity, the patient who arrived earlier must be treated first.

7.2 Why a normal FIFO queue is insufficient

A normal FIFO queue serves only by arrival order. It cannot automatically place a newly arriving critical patient ahead of a minor patient who arrived earlier. A PriorityQueue with a comparator can enforce severity priority and arrival order for ties.

7.3 Java program

```
import java.util.*;

class Patient {
    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    public String toString() {
        return name + "(S" + severity + ")";
    }
}

public class Triage {
    private long counter = 0;
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)
```

```
                .thenComparingLong(p -> p.arrival));

    void admit(String name, int severity) {
        if (severity < 1 || severity > 5)
            throw new IllegalArgumentException("Severity must be 1 to 5");
        queue.add(new Patient(name, severity, counter++));
    }

    Patient next() {
        return queue.poll();
    }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Latha", 2);

        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null) {
            if (order.length() > 0) order.append(' ');
            order.append(p);
        }
        System.out.println(order);
    }
}
```

7.4 Expected output

Meena(S1) Kumar(S1) Latha(S2) Ravi(S3) John(S3) Asha(S5)

7.5 Explanation of arrival counter

The counter increases for every admitted patient. Each Patient stores the counter value as its arrival number. The comparator first compares severity. If severity is equal, it compares arrival number. A smaller arrival number represents an earlier arrival, so the earlier patient is removed first.

7.6 Complexity

Operation	PriorityQueue complexity
Add / admit	O(log n)
Remove next patient / poll	O(log n)
Peek next patient	O(1)
Display all patients in priority order	Repeated removals: O(n log n)

7.7 Fairness and starvation

A patient with severity 5 may wait for a long time if critical patients continue to arrive. This is called starvation. A possible fairness rule is aging: after a patient waits beyond a defined time, the effective priority can be improved. Such a rule must be designed carefully so that urgent cases still receive appropriate treatment.

7.8 Alternative designs

A sorted LinkedList can insert each patient into the correct position, taking O(n) for admission and O(1) to remove the first patient. Five separate FIFO queues can provide O(1) admission and service within a severity level, but selecting the highest-priority non-empty queue requires checking up to five queues. The PriorityQueue provides a scalable general solution with O(log n) insertion and removal.

8. Q7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE

8.1 Selected solution

Selected task: Q2 – Music playlist. The original design uses ArrayList. It is efficient for indexed access and appending at the end, but frequent insertions and removals at the front require shifting many elements. This becomes a bottleneck when the application changes to a queue-like Up Next feature.

8.2 A. Diagnose the bottleneck

The main bottleneck is repeated insertion/removal at index 0 in an ArrayList. Each front insertion shifts existing elements one position to the right. Each front removal shifts remaining elements one position to the left. The amount of shifting grows with the number of songs.

8.3 B. Define the design objective

Redesign objective: support frequent additions at both ends and removal from the front without shifting the entire collection. Preserve the required order of songs and keep the program behaviour correct.

8.4 C. Design the revised solution

Use ArrayDeque<String> for the Up Next queue. addFirst() and addLast() add songs at either end, while removeFirst() serves the next song. ArrayDeque is designed for deque operations and does not require indexed access.

```
import java.util.*;

public class PlaylistRedesign {
    public static void main(String[] args) {
        ArrayDeque<String> upNext = new ArrayDeque<>();

        upNext.addLast("Song A");
        upNext.addLast("Song B");
        upNext.addLast("Song C");

        // Add an urgent song at the front
        upNext.addFirst("Priority Song");

        System.out.println("Queue before playing: " + upNext);
        System.out.println("Now playing: " + upNext.removeFirst());

        // Add a song at the end
        upNext.addLast("Song D");
        System.out.println("Remaining queue: " + upNext);
    }
}
```

8.5 D. Predict before implementing

Prediction: ArrayDeque should perform better than ArrayList for repeated front additions and removals as the collection grows, because deque operations at the ends are amortized O(1), while ArrayList front operations require O(n) shifting. ArrayList should remain better for frequent indexed reads.

8.6 E. Implement and validate

Run both implementations under the same conditions. Use at least three queue sizes. Repeat each workload several times, warm up the JVM and calculate an average. The following benchmark measures repeated front insertions and removals.

```
import java.util.*;

public class FrontOperationBenchmark {
    static long arrayListTest(int n) {
        ArrayList<Integer> list = new ArrayList<>();
        long start = System.nanoTime();
        for (int i = 0; i < n; i++) list.add(0, i);
        while (!list.isEmpty()) list.remove(0);
        return System.nanoTime() - start;
    }

    static long dequeTest(int n) {
        ArrayDeque<Integer> deque = new ArrayDeque<>();
        long start = System.nanoTime();
        for (int i = 0; i < n; i++) deque.addFirst(i);
        while (!deque.isEmpty()) deque.removeFirst();
        return System.nanoTime() - start;
    }

    public static void main(String[] args) {
        int[] sizes = {1000, 5000, 10000};
        for (int n : sizes) {
            // Warm-up
            arrayListTest(Math.min(n, 1000));
            dequeTest(Math.min(n, 1000));

            long a = 0, d = 0;
            int runs = 5;
            for (int r = 0; r < runs; r++) {
                a += arrayListTest(n);
                d += dequeTest(n);
            }
            System.out.println("n=" + n);
            System.out.println("ArrayList average ns=" + (a / runs));
            System.out.println("ArrayDeque average ns=" + (d / runs));
        }
    }
}
```

8.7 Experimental evidence

n	ArrayList avg (ns)	ArrayDeque avg (ns)	Prediction supported?	Notes
1,000	Fill after execution	Fill after execution	Yes / No	
5,000	Fill after execution	Fill after execution	Yes / No	
10,000	Fill after execution	Fill after execution	Yes / No	

8.8 F. Defend the final design

If the measured results show ArrayDeque is faster for front operations, the prediction is supported. The redesign is appropriate for an Up Next queue because the workload is dominated by operations at the ends. However, ArrayDeque does not provide efficient indexed access and is not a replacement for ArrayList in every playlist feature. If users frequently jump to the k-th song, ArrayList may remain preferable for the main playlist.

8.9 Trade-offs and deployment decision

- Benefit: efficient operations at both ends; avoids repeated shifting of the full array for front changes.
- Trade-off: no direct indexed access; null elements are not permitted.
- Memory: ArrayDeque uses a resizable array internally; it does not allocate one linked node per element.
- Deployment: use ArrayDeque for the Up Next queue and retain ArrayList for the main indexed playlist if both access patterns are required.

A redesign that does not outperform the original in every measurement is not automatically incorrect. The final decision must match the actual workload and should be supported by the measured evidence.

9. OVERALL CONCLUSION

This assignment demonstrates that selecting a Java data structure requires understanding the operations performed by the application. StringBuilder is generally suitable for repeated single-threaded text construction. ArrayList is useful for indexed access, while ArrayDeque is suitable for queue operations at both ends. HashSet requires a correct equals() and hashCode() implementation to avoid duplicate logical objects. HashMap, TreeMap and LinkedHashMap provide different lookup and ordering behaviours. Comparator chains make ranking rules reusable, and PriorityQueue supports severity-based service with arrival-order tie-breaking.

The Day-4 redesign shows that a collection should be selected based on measured workload rather than a general statement that one class is always faster. The original and redesigned solutions must be tested under the same conditions, and the final engineering decision must explain benefits and trade-offs.

CODE:

Q1 – TicketTextBenchmark.java

```
public class TicketTextBenchmark {

static String buildString(int n) {

String s = "";

for (int i = 1; i <= n; i++) {

s = s + "Field" + i + ":Value" + i + "\n";

}

return s;

}

static String buildBuilder(int n) {

StringBuilder s = new StringBuilder();

for (int i = 1; i <= n; i++) {

s.append("Field").append(i).append(":Value").append(i).append("\n");

}
```

```
return s.toString();
}
static String buildBuffer(int n) {
StringBuffer s = new StringBuffer();
for (int i = 1; i <= n; i++) {
s.append("Field").append(i).append(":Value").append(i).append("\n");
}
return s.toString();
}
static long measure(Runnable task, int runs) {
long start = System.nanoTime();
for (int i = 0; i < runs; i++) {
task.run();
}
return (System.nanoTime() - start) / runs;
}
public static void main(String[] args) {
int n = 50;
int runs = 2000;
System.out.println("String average ns: " + measure(() -> buildString(n), runs));
System.out.println("StringBuilder average ns: " + measure(() -> buildBuilder(n),
runs));
System.out.println("StringBuffer average ns: " + measure(() -> buildBuffer(n),
runs));
System.out.println("Ticket Preview:");
```

```
System.out.print(buildBuilder(5));
}
}
```

Q2 – PlaylistComparison.java

```
import java.util.*;
public class PlaylistComparison {
static long testArrayList(int n) {
ArrayList<Integer> list = new ArrayList<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
list.add(i);
}
for (int i = 0; i < n; i++) {
list.get(i);
}
return System.nanoTime() - start;
}
static long testLinkedList(int n) {
LinkedList<Integer> list = new LinkedList<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
list.add(i);
}
```

```
for (int i = 0; i < n; i++) {
list.get(i);
}
return System.nanoTime() - start;
}
public static void main(String[] args) {
int[] sizes = {1000, 5000, 10000};
for (int n : sizes) {
System.out.println("n = " + n);
System.out.println("ArrayList ns = " + testArrayList(n));
System.out.println("LinkedList ns = " + testLinkedList(n));
}
ArrayDeque<String> queue = new ArrayDeque<>();
queue.addLast("Song A");
queue.addLast("Song B");
queue.addFirst("Priority Song");
System.out.println("Next song: " + queue.removeFirst());
}
}
```

Q3 – AttendanceDemo.java

```
import java.util.*;
class Student {
String regNo, name;
Student(String regNo, String name) {
this.regNo = regNo;
this.name = name;
}
public boolean equals(Object obj) {
if (this == obj) return true;
if (!(obj instanceof Student)) return false;
Student s = (Student) obj;
return Objects.equals(regNo, s.regNo);
}
public int hashCode() {
return Objects.hash(regNo);
}
public String toString() {
return regNo + " " + name;
}
}
public class AttendanceDemo {
public static void main(String[] args) {
Set<Student> students = new HashSet<>();
```

```
students.add(new Student("192311001", "Arun"));
students.add(new Student("192311002", "Divya"));
students.add(new Student("192311001", "Arun"));
System.out.println("Unique students: " + students.size());
for (Student s : students) {
System.out.println(s);
}
}
}
```

Q4 – PharmacyInventory.java

```
import java.util.*;
public class PharmacyInventory {
public static void main(String[] args) {
Map<String, Integer> hash = new HashMap<>();
Map<String, Integer> tree = new TreeMap<>();
Map<String, Integer> linked = new LinkedHashMap<>();
String[] codes = {"M103", "M101", "M105", "M102", "M104"};
int[] stocks = {40, 25, 60, 30, 15};
for (int i = 0; i < codes.length; i++) {
hash.put(codes[i], stocks[i]);
tree.put(codes[i], stocks[i]);
linked.put(codes[i], stocks[i]);
}
```

```
System.out.println("Lookup M103: " + hash.get("M103"));
System.out.println("Sorted code report: " + tree);
System.out.println("Order-added report: " + linked);
System.out.println("HashMap report: " + hash);
}
}
```

Q5 – LeaderboardDemo.java

```
import java.util.*;
class Team {
String name;
int points, goalDiff;
Team(String name, int points, int goalDiff) {
this.name = name;
this.points = points;
this.goalDiff = goalDiff;
}
public String toString() {
return name + "(" + points + "," + goalDiff + ")";
}
}
class Leaderboard<T> {
List<T> items = new ArrayList<>();
Comparator<? super T> order;
```

```
Leaderboard(Comparator<? super T> order) {
this.order = order;
}
void add(T item) {
items.add(item);
}
List<T> ranking() {
List<T> result = new ArrayList<>(items);
result.sort(order);
return result;
}
}
public class LeaderboardDemo {
public static void main(String[] args) {
Comparator<Team> rank = Comparator.comparingInt((Team t) -> t.points)
.reversed()
.thenComparing(Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
.thenComparing(t -> t.name);
Leaderboard<Team> board = new Leaderboard<>(rank);
board.add(new Team("Panthers", 9, 4));
board.add(new Team("Eagles", 9, 6));
board.add(new Team("Cobras", 6, 1));
board.add(new Team("Bisons", 9, 4));
System.out.println(board.ranking());
```

```
}
}
```

Q6 – Triage.java

```
import java.util.*;
class Patient {
String name;
int severity;
long arrival;
Patient(String name, int severity, long arrival) {
this.name = name;
this.severity = severity;
this.arrival = arrival;
}
public String toString() {
return name + "(S" + severity + ")";
}
}
public class Triage {
long counter = 0;
PriorityQueue<Patient> queue = new PriorityQueue<>(
Comparator.comparingInt((Patient p) -> p.severity)
.thenComparingLong(p -> p.arrival)
);
```

```
void admit(String name, int severity) {
if (severity < 1 || severity > 5) {
throw new IllegalArgumentException("Severity must be 1 to 5");
}
queue.add(new Patient(name, severity, counter++));
}
public static void main(String[] args) {
Triage er = new Triage();
er.admit("Ravi", 3);
er.admit("Meena", 1);
er.admit("John", 3);
er.admit("Asha", 5);
er.admit("Kumar", 1);
er.admit("Latha", 2);
Patient p;
StringBuilder result = new StringBuilder();
while ((p = er.queue.poll()) != null) {
if (result.length() > 0) result.append(" ");
result.append(p);
}
System.out.println(result);
}
}
```

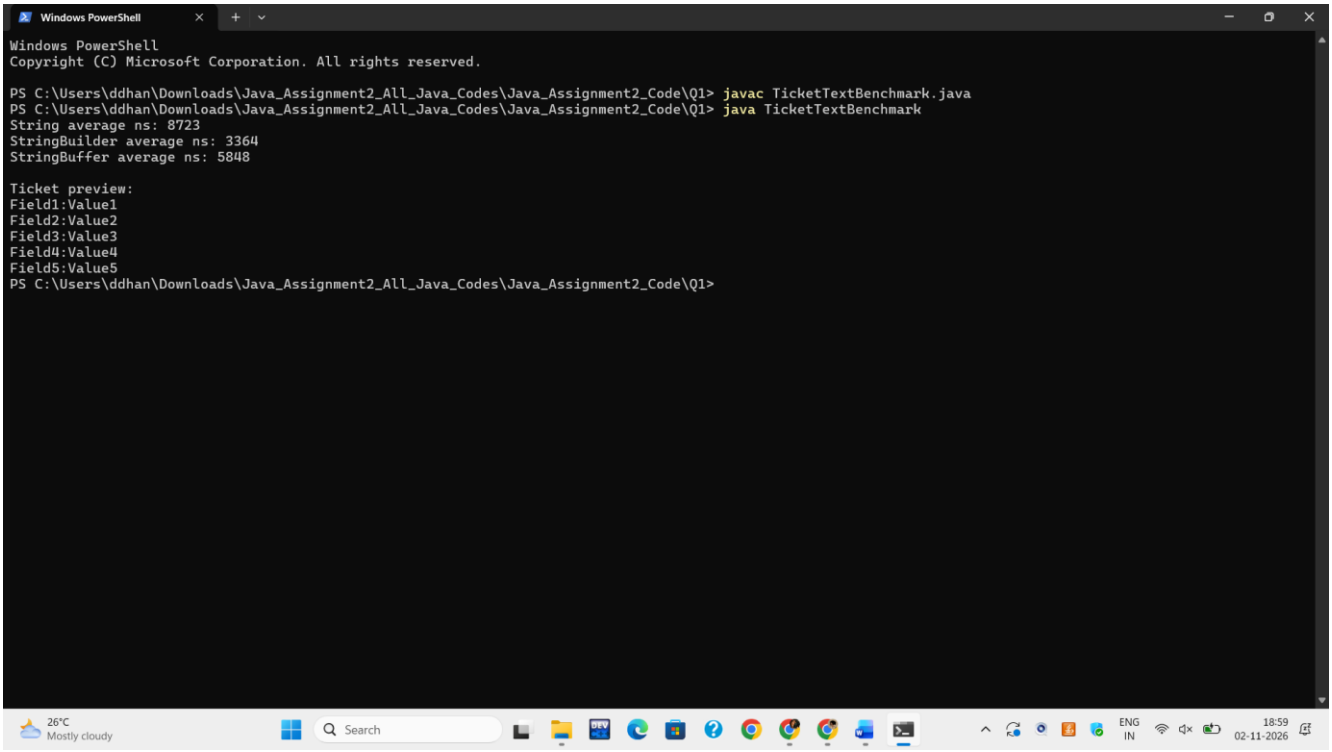
Q7 – FrontOperationBenchmark.java

```
import java.util.*;
public class FrontOperationBenchmark {
static long testArrayList(int n) {
ArrayList<Integer> list = new ArrayList<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
list.add(0, i);
}
while (!list.isEmpty()) {
list.remove(0);
}
return System.nanoTime() - start;
}
static long testArrayDeque(int n) {
ArrayDeque<Integer> deque = new ArrayDeque<>();
long start = System.nanoTime();
for (int i = 0; i < n; i++) {
deque.addFirst(i);
}
while (!deque.isEmpty()) {
deque.removeFirst();
}
return System.nanoTime() - start;
```

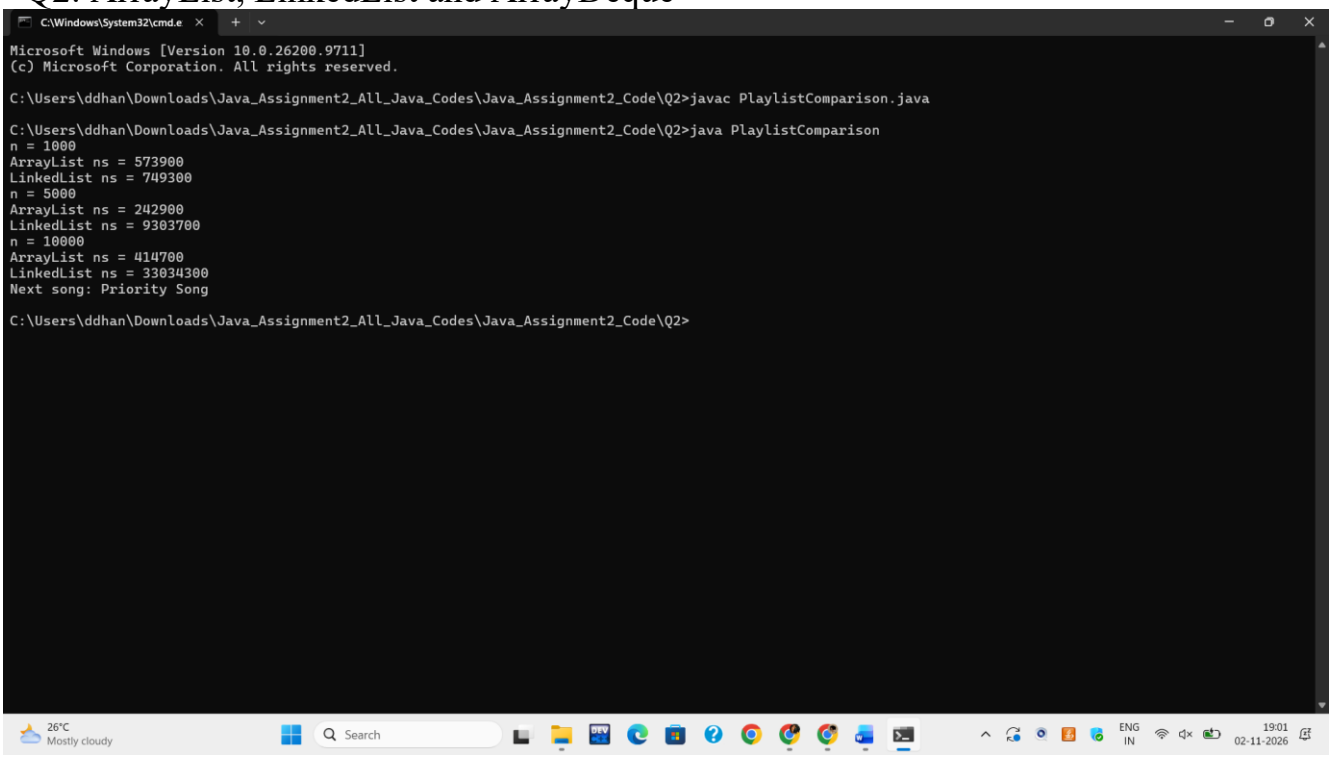
```
}
public static void main(String[] args) {
int[] sizes = {1000, 5000, 10000};
int runs = 5;
for (int n : sizes) {
testArrayList(Math.min(n, 1000));
testArrayDeque(Math.min(n, 1000));
long a = 0, d = 0;
for (int i = 0; i < runs; i++) {
a += testArrayList(n);
d += testArrayDeque(n);
}
System.out.println("n = " + n);
System.out.println("ArrayList average ns = " + (a / runs));
System.out.println("ArrayDeque average ns = " + (d / runs));
}
}
}
```

APPENDIX A – OUTPUT

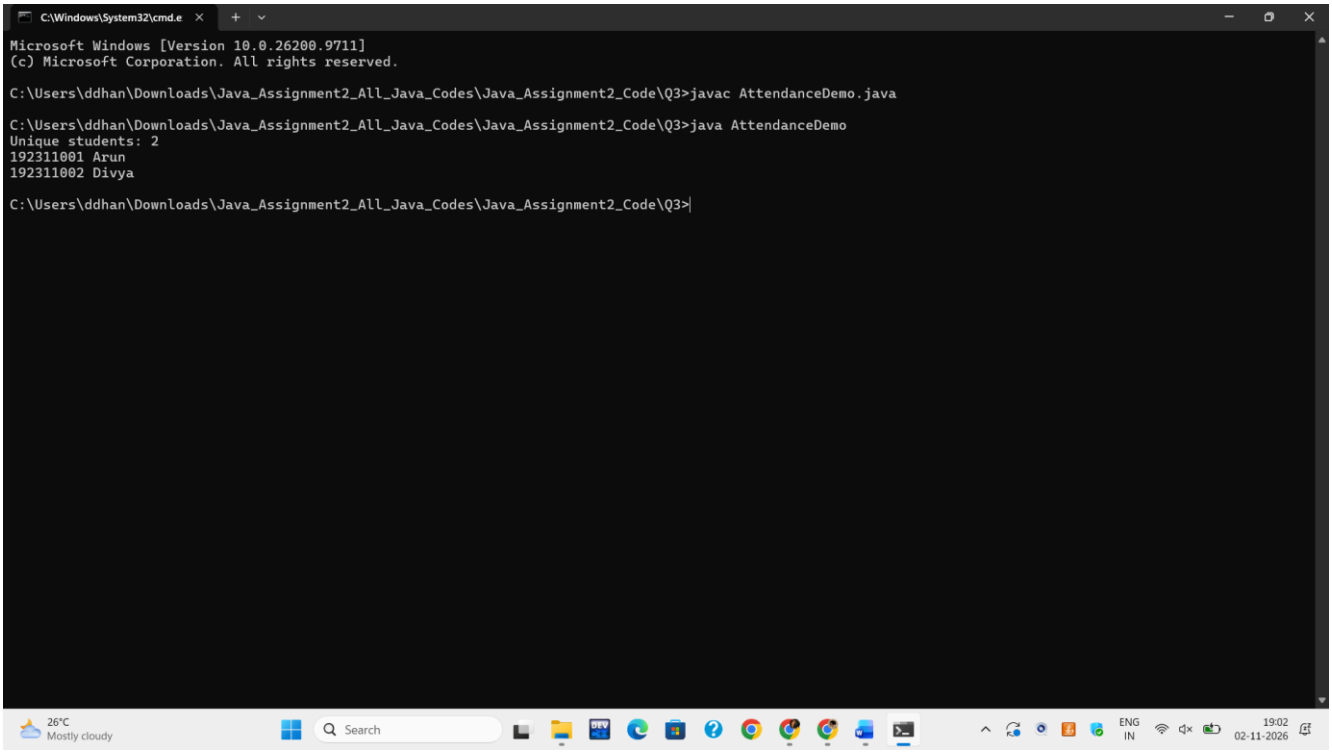
- Q1: String, StringBuilder and StringBuffer



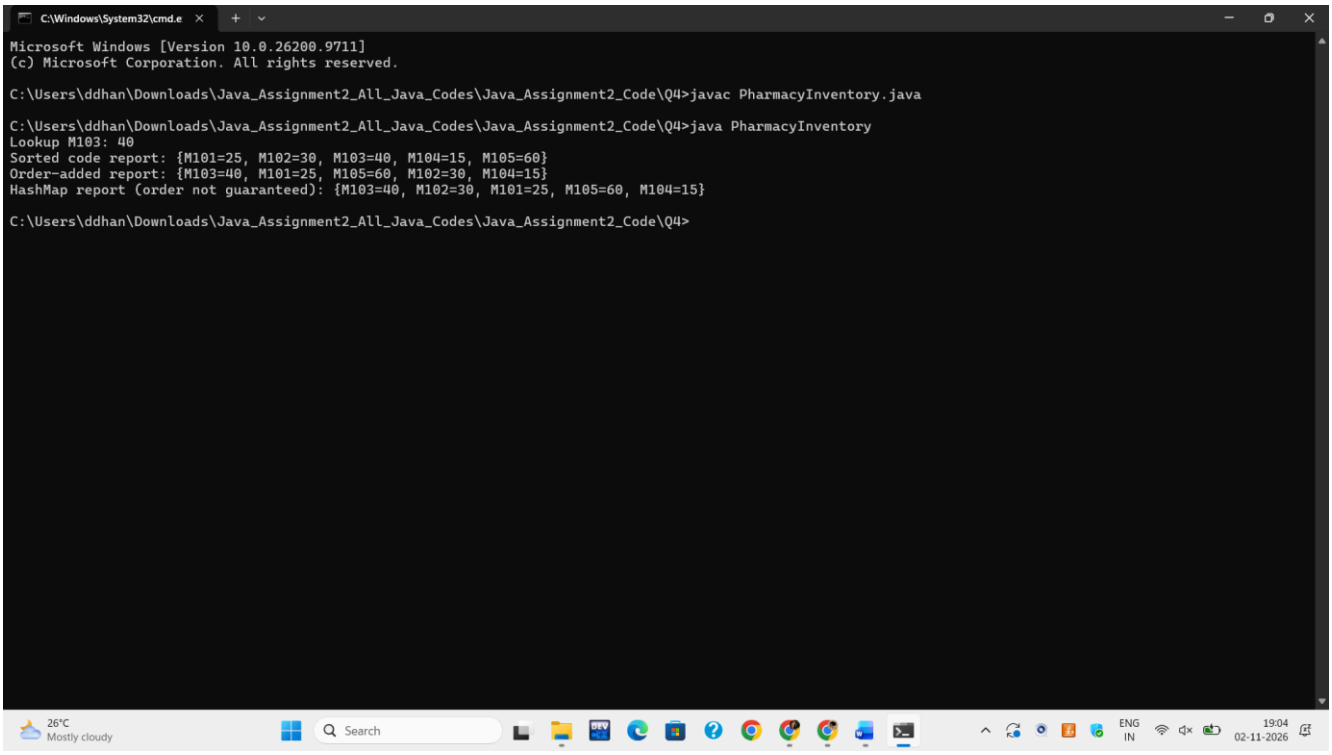
- Q2: ArrayList, LinkedList and ArrayDeque



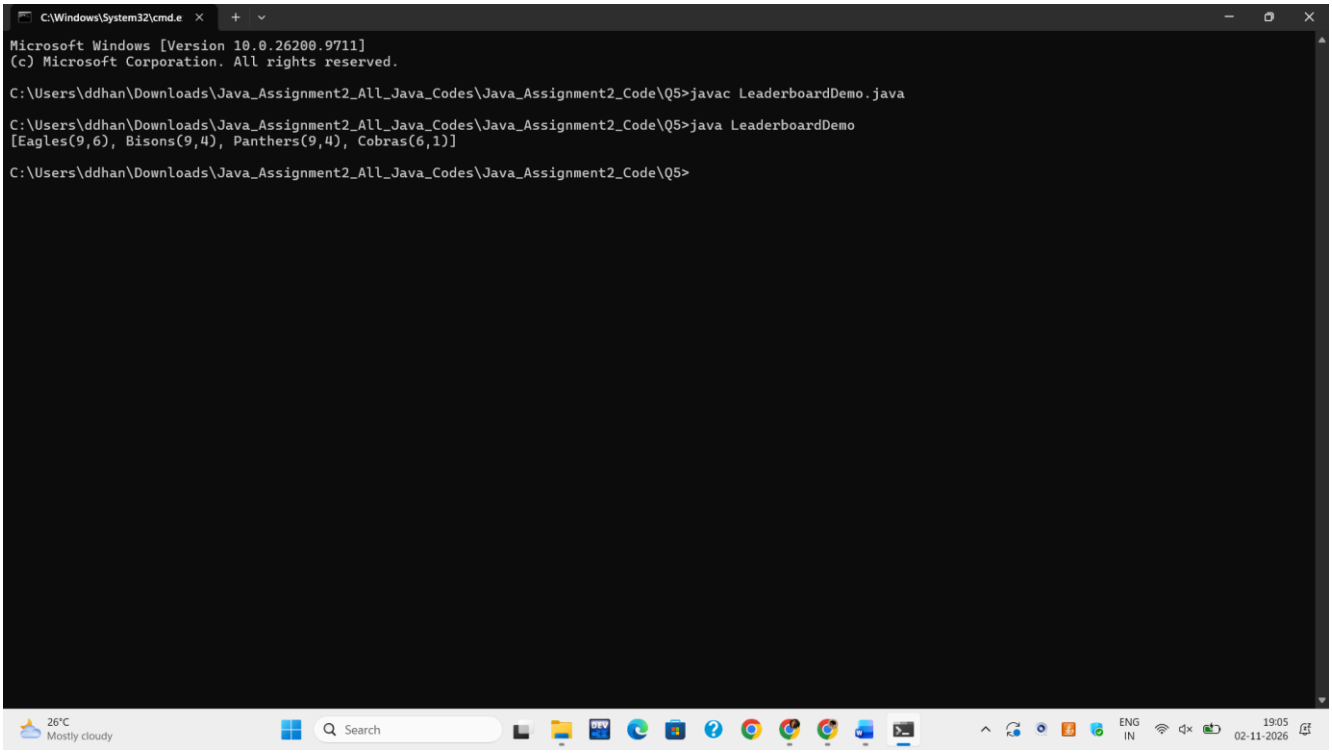
- Q3: HashSet and hashCode-equals



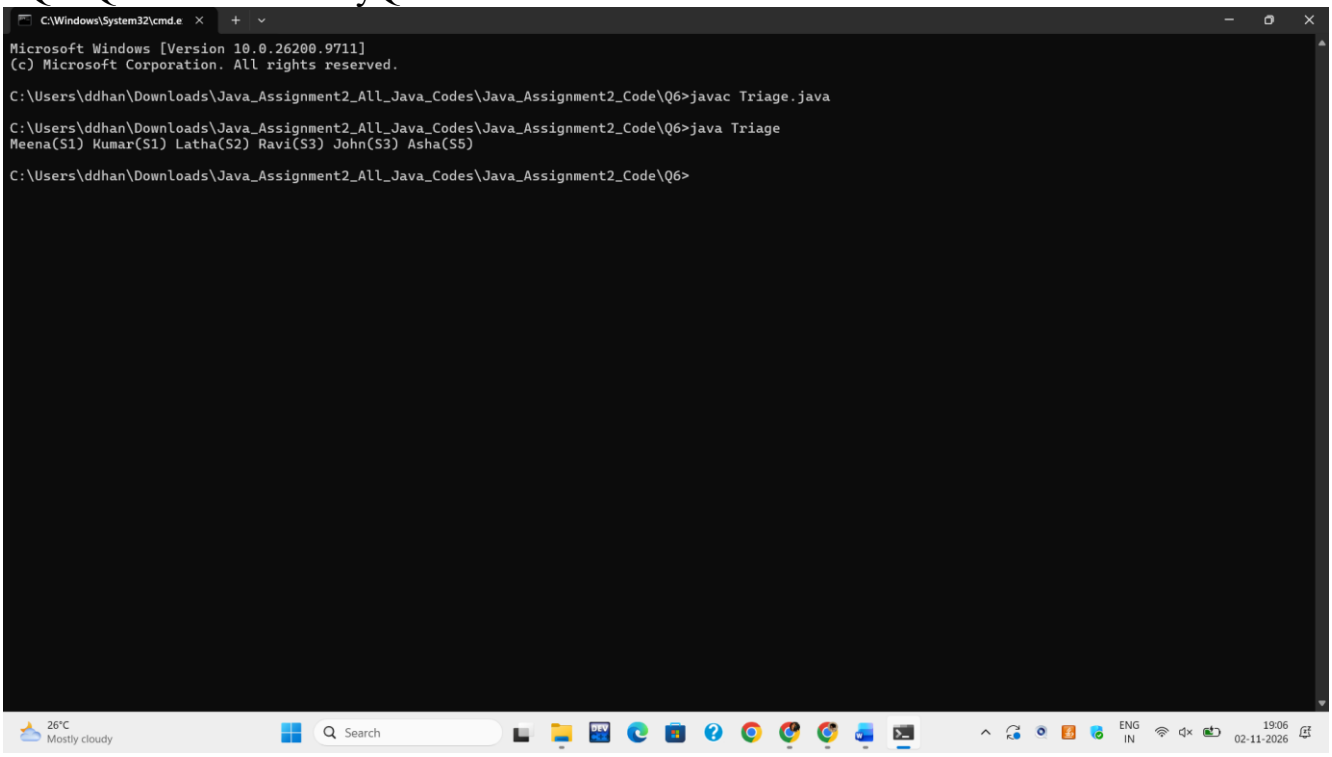
- Q4: HashMap, TreeMap and LinkedHashMap



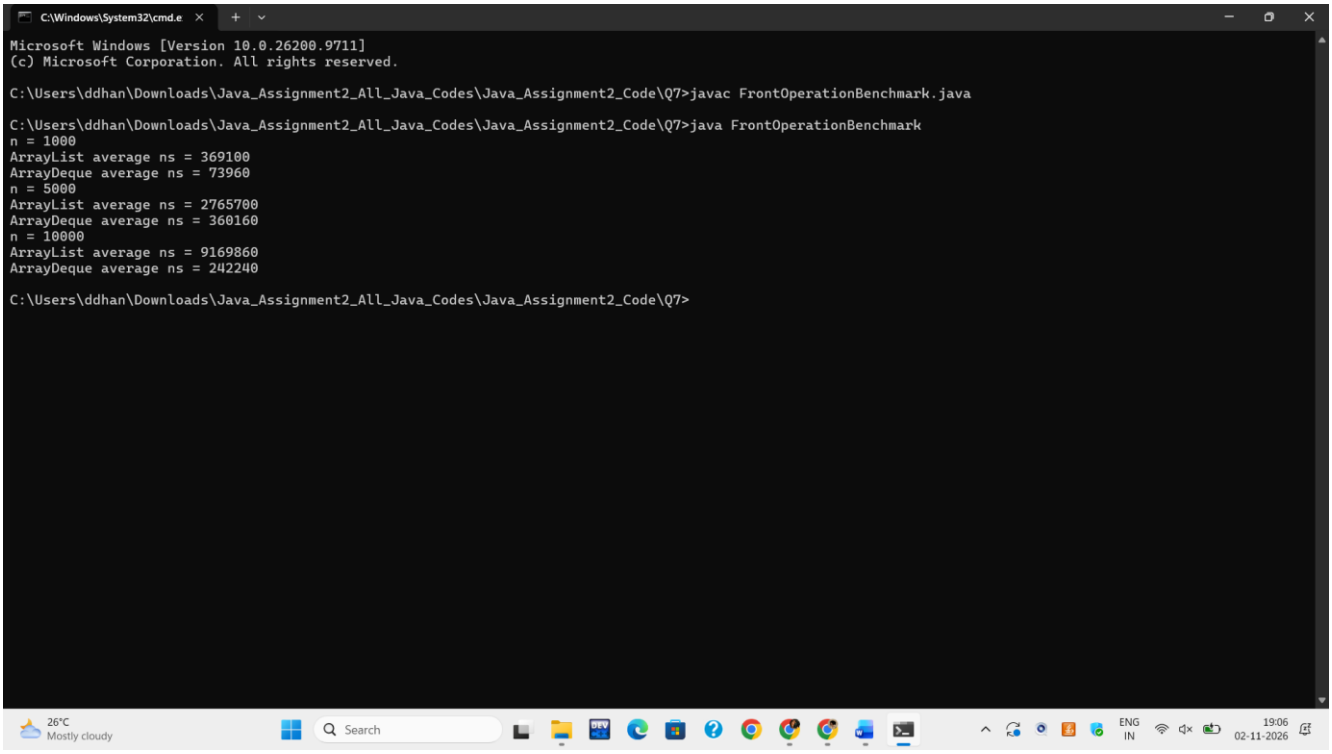
- Q5: Generics, Comparable and Comparator



- Q6: Queue and PriorityQueue



- Q7: Diagnose, redesign and validate



APPENDIX B – IMPORTANT VIVA QUESTIONS

What is an immutable String?

A String cannot be changed after it is created. Operations that modify it create another String.

Why is StringBuilder used?

It builds text by modifying a buffer and is efficient for repeated concatenation in one thread.

Difference between ArrayList and LinkedList?

ArrayList supports fast indexed access; LinkedList supports efficient insertion/removal at its ends.

Why override hashCode() with equals()?

Equal objects must have the same hash code so hash-based collections can find duplicates correctly.

Difference between HashMap and TreeMap?

HashMap has no guaranteed key order; TreeMap keeps keys sorted.

What is a Comparator?

An object that defines an external ordering between objects.

Why use PriorityQueue in triage?

It removes the patient with the highest priority according to the comparator.

What is starvation?

A low-priority item may wait indefinitely if higher-priority items keep arriving.

What is Big-O notation?

It describes how an algorithm's resource use grows as input size increases.

Why measure more than one input size?

To understand how performance changes as the amount of data grows.